

Mosher Lawns Time Tracker - Complete Repository Walkthrough

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [Directory Structure](#)
 4. [Core Data Layer](#)
 5. [Frontend Architecture](#)
 6. [Backend Architecture](#)
 7. [Key Screens & Features](#)
 8. [Styling & Theme System](#)
 9. [Configuration & Build](#)
 10. [Testing & Quality](#)
-

Project Overview

Mosher Lawns Time Tracker is a React Native mobile application built with Expo that enables time tracking for lawn care company employees. The app allows:

- **Employees** to clock in/out for jobs
- **Supervisors** to manage employees and job sites
- **Data Management** to view, edit, and export time logs
- **CSV Export** for payroll and reporting
- **Multi-employee Selection** for bulk clocking in
- **Log Grouping** for condensed readability when multiple employees work the same job

The app is designed as a **shared-device interface** with no user authentication—anyone can use it to track time.

Key Features

- ✓ Clock in/out multiple employees simultaneously
 - ✓ Manage employees and job sites with full CRUD operations
 - ✓ View grouped time logs with employee badges
 - ✓ Manually add or edit past time entries
 - ✓ Export time logs to CSV format
 - ✓ Light/dark theme support
 - ✓ Local data persistence with AsyncStorage
 - ✓ Optional backend server with OAuth authentication (not currently used)
-

Technology Stack

Frontend (React Native)

Technology	Version	Purpose
React Native	0.81.5	Core mobile framework
Expo	54.0.29	Development platform & build system
Expo Router	6.0.19	File-based routing (like Next.js)
React	19.1.0	UI component library
TypeScript	5.9.3	Static type checking
NativeWind	4.2.1	Tailwind CSS for React Native
React Native Reanimated	4.1.6	Smooth animations
AsyncStorage	2.2.0	Local data persistence

Backend (Optional)

Technology	Version	Purpose
Express	4.22.1	HTTP server framework
tRPC	11.7.2	Type-safe API layer
Drizzle ORM	0.44.7	Database query builder
MySQL	3.16.0	Database driver

Development Tools

Tool	Version	Purpose
pnpm	9.12.0	Package manager
Vitest	2.1.9	Unit testing framework
ESLint	9.39.2	Code linting
Prettier	3.7.4	Code formatting
Tailwind CSS	3.4.17	Styling engine

Directory Structure

Root Level Files

```
mosher-lawns-time-tracker/
├── app.config.ts           # Expo app configuration (bundle ID, icon, etc.)
├── package.json            # Project dependencies and scripts
├── tsconfig.json           # TypeScript configuration
├── tailwind.config.js      # Tailwind CSS configuration
├── theme.config.js         # Theme color tokens (light/dark)
├── babel.config.js         # Babel transpiler configuration
├── metro.config.js         # Metro bundler configuration
├── drizzle.config.ts       # Database migration configuration
├── eslint.config.js        # ESLint rules
├── design.md              # UI/UX design document
├── todo.md                # Project task tracking
├── README.md              # Template documentation
└── template.json          # Template metadata
```

/app - Mobile App Screens & Navigation

```
app/
├── _layout.tsx           # Root layout (providers, initialization)
├── login.tsx             # Stub file (auth removed for shared dev)
├── data-management.tsx   # Hub screen for admin functions
├── employees-management.tsx # Employee CRUD screen
├── jobs-management.tsx   # Job site CRUD screen
├── edit-log.tsx          # Manual time entry add/edit screen
├── (tabs)/               # Tab bar navigation group
│   ├── _layout.tsx      # Tab bar configuration
│   ├── index.tsx        # Clock In/Out screen (HOME TAB)
│   └── logs.tsx         # Time Logs view (LOGS TAB)
├── oauth/
│   └── callback.tsx      # OAuth callback handler (not used)
└── dev/
    └── theme-lab.tsx     # Development theme testing screen
```

Key Concept: The `(tabs)` folder uses Expo Router's **route groups** feature. Parentheses indicate a folder that doesn't appear in the URL but groups related screens under a tab bar.

/components - Reusable UI Components

```
components/
├── screen-container.tsx  # SafeArea wrapper for all screens
├── themed-view.tsx       # View with theme-aware background
├── haptic-tab.tsx        # Tab bar with haptic feedback
├── hello-wave.tsx        # Welcome animation component
├── parallax-scroll-view.tsx # Parallax scrolling container
├── external-link.tsx     # Link component for web
└── ui/
    ├── icon-symbol.tsx   # Icon mapping (SF Symbols → Material Icons)
    ├── icon-symbol.ios.tsx # iOS-specific icon implementation
    └── collapsible.tsx   # Collapsible/accordion component
```

ScreenContainer is the most important component—it wraps all screens to handle:

- Safe area insets (notches, home indicators)
- Background color extension
- Proper tab bar spacing

/lib - Core Libraries & Utilities

```
lib/
├── local-storage.ts      # ★ CORE DATA LAYER - All CRUD operations
├── trpc.ts               # trpc client configuration
├── utils.ts              # Utility functions (cn, classname merging)
├── theme-provider.tsx    # Theme context & light/dark mode
└── _core/
    ├── theme.ts          # Theme builder (colors, fonts)
    ├── auth.ts           # Authentication helpers
    ├── api.ts            # API client initialization
    ├── manus-runtime.ts  # Manus platform integration
    └── nativewind-pressable.ts # NativeWind configuration
```

local-storage.ts is the heart of the app—it defines:

- Data models (Employee, JobSite, TimeLog)
- All CRUD operations
- CSV export logic
- Grouping logic

/hooks - React Hooks

```
hooks/
├── use-colors.ts        # Get current theme colors
├── use-color-scheme.ts  # Detect light/dark mode (native)
├── use-color-scheme.web.ts # Detect light/dark mode (web)
├── use-auth.ts          # Authentication state (unused)
└── use-local-auth.ts    # Local auth helpers (unused)
```

/constants - Configuration Constants

```
constants/  
├── theme.ts           # Re-export theme colors  
├── oauth.ts          # OAuth URLs and deep-link schemes  
└── const.ts         # App constants
```

/server - Backend API (Optional)

```
server/  
├── README.md          # Backend documentation  
├── routers.ts         # ★ tRPC API routes definition  
├── db.ts              # Database helper functions  
├── storage.ts         # S3 storage proxy  
└── _core/  
    ├── index.ts      # Server startup & Express middleware  
    ├── trpc.ts       # tRPC setup & auth middleware  
    ├── context.ts    # tRPC request context  
    ├── oauth.ts      # OAuth route handlers  
    ├── cookies.ts    # Session cookie management  
    ├── auth.ts       # Authentication utilities  
    ├── sdk.ts        # Manus SDK integration  
    ├── dataApi.ts    # Data API wrapper  
    ├── imageGeneration.ts # AI image generation  
    ├── llm.ts        # LLM/AI integration  
    ├── notification.ts # Push notifications  
    ├── voiceTranscription.ts # Speech-to-text  
    ├── heartbeat.ts  # Server health check  
    ├── storageProxy.ts # S3 proxy routes  
    ├── systemRouter.ts # System tRPC routes  
    └── types/  
        ├── manusTypes.ts # Manus platform types  
        └── cookie.d.ts    # Cookie type definitions
```

/drizzle - Database Migrations

```
drizzle/
├── schema.ts           # ★ Database table definitions
├── relations.ts        # Table relationships
├── 0000_elite_eternals.sql  # Initial migration
├── 0001_overjoyed_proteus.sql # Second migration
└── meta/
    ├── _journal.json    # Migration history
    ├── 0000_snapshot.json # Schema snapshot
    └── 0001_snapshot.json # Schema snapshot
```

/shared - Shared Code (Frontend & Backend)

```
shared/
├── types.ts           # Shared TypeScript types
├── const.ts           # Shared constants
└── _core/
    └── errors.ts      # Error definitions
```

/assets - Images & Icons

```
assets/
└── images/
    ├── icon.png       # App launcher icon
    ├── splash-icon.png # Splash screen icon
    ├── favicon.png     # Web favicon
    ├── android-icon-*.png # Android adaptive icons
    └── react-logo*.png  # Template images
```

/scripts - Build & Setup Scripts

```
scripts/  
├─ load-env.js           # Environment variable loader  
├─ generate_qr.mjs       # QR code generator for Expo  
└─ reset-project.js     # Project reset utility
```

/tests - Unit Tests

```
tests/  
└─ auth.logout.test.ts   # Authentication test suite
```

Core Data Layer

File: `lib/local-storage.ts`

This is the **most important file** in the entire app. It contains:

1. Data Models (TypeScript Interfaces)

```
interface Employee {
  id: string;           // Unique identifier (Date.now().toString())
  name: string;         // Employee's full name
}

interface JobSite {
  id: string;           // Unique identifier
  name: string;         // Job site name
  location: string;     // Address/location
}

interface TimeLog {
  id: string;           // Unique identifier
  employeeId: string;   // Reference to Employee
  jobSiteId: string;    // Reference to JobSite
  clockInTime: number;  // Unix timestamp (milliseconds)
  clockOutTime: number | null; // Unix timestamp or null if still clocked in
}
```

Key Design Decisions:

- IDs are generated using `Date.now().toString()` for simplicity
- `clockOutTime: null` indicates an active/ongoing shift
- All timestamps are Unix milliseconds for easy calculation

2. Storage Keys

```
const STORAGE_KEYS = {
  EMPLOYEES: 'employees',
  JOB_SITES: 'job_sites',
  TIME_LOGS: 'time_logs',
};
```

These keys are used to store data in AsyncStorage (React Native's local storage).

3. Employee Management Functions

getEmployees(): Promise<Employee[]>

- Retrieves all employees from AsyncStorage
- Returns empty array if no data exists
- Handles errors gracefully

addEmployee(name: string): Promise<Employee | null>

- Creates a new employee with unique ID
- Prevents duplicate names (case-insensitive)
- Returns the created employee or null if duplicate

updateEmployee(id: string, name: string): Promise<void>

- Updates an employee's name by ID
- Finds employee by ID and updates in-place

deleteEmployee(employeeId: string): Promise<void>

- Removes employee from storage
- Does NOT delete associated time logs (orphaned records remain)

4. Job Site Management Functions

getJobSites(): Promise<JobSite[]>

- Retrieves all job sites from AsyncStorage

addJobSite(name: string, location: string): Promise<JobSite | null>

- Creates a new job site
- Prevents duplicate names

updateJobSite(id: string, name: string, location: string): Promise<void>

- Updates job site details

deleteJobSite(jobSiteId: string): Promise<void>

- Removes job site from storage

5. Time Log Management Functions

getTimeLogs(): Promise<TimeLog[]>

- Retrieves all time logs

addTimeLog(employeeId: string, jobSiteId: string, clockInTime: number): Promise<TimeLog>

- Creates a new time log entry
- Sets `clockOutTime: null` (active shift)

clockOutTimeLog(timeLogId: string): Promise<void>

- Marks a time log as complete
- Sets `clockOutTime` to current time

updateTimeLog(id: string, updates: Partial<TimeLog>): Promise<void>

- Allows manual editing of time log details
- Used for correcting past entries

deleteTimeLog(timeLogId: string): Promise<void>

- Removes a time log entry

deleteAllTimeLogs(): Promise<void>

- Clears all time logs (used for testing/reset)

6. Advanced Features

exportLogsToCSV(): Promise<string>

- Generates CSV format of all time logs
- Includes: Employee, Job Site, Clock In, Clock Out, Duration (hours)
- Formats dates in user's locale
- Returns CSV string ready for file export

initializeDefaultData(): Promise<void>

- Runs once on app startup

- Creates sample employees and job sites if none exist
- Ensures app has data to work with

getDailyReport(date: Date): Promise<DailyReport>

- Calculates total hours worked per employee on a given date
- Used for daily summaries

getEmployeeSummary(employeeId: string): Promise<EmployeeSummary>

- Calculates total hours for a specific employee
- Used for employee profiles

7. Data Flow Example

Clocking In Multiple Employees:

```
User selects employees [Alice, Bob] and job "Site A"
↓
User taps "Clock In"
↓
App calls: addTimeLog("alice-id", "site-a-id", Date.now())
App calls: addTimeLog("bob-id", "site-a-id", Date.now())
↓
Both entries stored in AsyncStorage with clockOutTime: null
↓
UI updates to show both employees as "Currently Clocked In"
```

Clocking Out:

```
User taps "Clock Out" on Alice's entry
↓
App calls: clockOutTimeLog("alice-log-id")
↓
App finds the log and sets clockOutTime to Date.now()
↓
Duration calculated: (clockOutTime - clockInTime) / (1000 * 60 * 60) = hours
↓
UI updates to show Alice as "Clock Out" with duration
```

Frontend Architecture

File: `app/_layout.tsx` - Root Layout

This is the **entry point** of the entire app. It:

1. Prevents splash screen from auto-hiding

```
SplashScreen.preventAutoHideAsync();
```

2. Initializes default data on app startup

```
useEffect(() => {
  async function prepare() {
    await initializeDefaultData(); // Seed sample data
    SplashScreen.hideAsync();      // Show app
  }
  prepare();
}, []);
```

3. Wraps entire app in providers

```

<GestureHandlerRootView>           // Gesture recognition
  <SafeAreaProvider>                // Safe area insets
    <ThemeProvider>                 // Light/dark mode
      <Stack>                       // Navigation
        <Stack.Screen name="(tabs)" />
        <Stack.Screen name="data-management" />
      </Stack>
    </ThemeProvider>
  </SafeAreaProvider>
</GestureHandlerRootView>

```

Provider Hierarchy:

- `GestureHandlerRootView` - Enables gesture recognition (swipes, pinches)
- `SafeAreaProvider` - Provides safe area insets (notches, home indicators)
- `ThemeProvider` - Manages light/dark mode globally
- `Stack` - Expo Router navigation (file-based routing)

File: `app/(tabs)/_layout.tsx` - Tab Navigation

Configures the bottom tab bar with two tabs:

```

<Tabs screenOptions={{ headerShown: false }}>
  <Tabs.Screen name="index" options={{ title: 'Clock In/Out' }} />
  <Tabs.Screen name="logs" options={{ title: 'Logs' }} />
</Tabs>

```

File: `app/(tabs)/index.tsx` - Clock In/Out Screen

Purpose: Main screen where employees clock in/out

Key Features:

- Multi-select employee list with checkboxes
- Job site dropdown selector
- “Clock In” button (creates new time logs for all selected employees)
- “Clock Out” button (completes active time logs)

- Shows active clocks with elapsed time
- Quick links to management screens

Data Flow:

```
Load employees, jobs, and active time logs on screen focus
↓
User selects employees and job site
↓
User taps "Clock In"
↓
For each selected employee:
  - Create new TimeLog with clockInTime = now
  - Store in AsyncStorage
↓
UI updates to show active clocks
```

File: `app/(tabs)/logs.tsx` - Time Logs Screen

Purpose: View all time logs with advanced features

Key Features:

- **Grouping Logic:** Groups logs by (jobSiteId, clockInTime, clockOutTime)
 - Shows multiple employees on same job at same time in one card
 - Displays employee count badge
 - Shows all employee names
- **Condensed Display:** Reduces visual clutter when multiple employees work together
- **Edit/Delete Actions:** Per-log or per-group operations
- **CSV Export:** Downloads all logs as CSV file
- **Manage Links:** Quick access to add/edit screens

Grouping Example:

Raw Logs:

- Alice clocked in at 9:00 AM, clocked out at 5:00 PM at Site A
- Bob clocked in at 9:00 AM, clocked out at 5:00 PM at Site A
- Charlie clocked in at 10:00 AM, clocked out at 3:00 PM at Site B

Grouped Display:

Site A - 9:00 AM to 5:00 PM

 2 employees

Alice, Bob

Duration: 8.0 hours

Site B - 10:00 AM to 3:00 PM

 1 employee

Charlie

Duration: 5.0 hours

File: `app/employees-management.tsx` - Employee CRUD

Purpose: Manage employee list

Features:

- **Add Employee:** Form to create new employee
- **Edit Employee:** Inline edit mode for each employee
- **Delete Employee:** Confirmation dialog before deletion
- **List Display:** FlatList of all employees with edit/delete buttons

Data Flow:

```
Load employees from AsyncStorage
↓
User enters name and taps "Add"
↓
Check for duplicates
↓
Create new Employee with unique ID
↓
Save to AsyncStorage
↓
Refresh UI
```

File: `app/jobs-management.tsx` - Job Site CRUD

Purpose: Manage job site list

Features:

- **Add Job Site:** Form with name and location fields
- **Edit Job Site:** Inline edit mode
- **Delete Job Site:** With confirmation
- **List Display:** Shows all job sites with location

File: `app/edit-log.tsx` - Manual Time Entry

Purpose: Add or edit time log entries manually

Features:

- **Multi-employee Selection:** Select multiple employees for one entry
 - Creates separate log for each employee
 - Useful for bulk manual entries
- **Date/Time Pickers:** Select clock in and clock out times
- **Manual Clock-Out Toggle:** Option to manually set clock out time
- **Edit Mode:** When editing existing log, shows current values

Multi-Employee Selection Example:

```
User selects: Alice, Bob, Charlie
User selects: Site A
User sets: Clock In = 9:00 AM, Clock Out = 5:00 PM
User taps "Save"
```

↓

```
Creates 3 separate logs:
- Alice at Site A, 9:00 AM - 5:00 PM
- Bob at Site A, 9:00 AM - 5:00 PM
- Charlie at Site A, 9:00 AM - 5:00 PM
```

File: `app/data-management.tsx` - Admin Hub

Purpose: Navigation hub for admin functions

Features:

- Button to navigate to Employees screen
- Button to navigate to Jobs screen
- Back button

Styling & Theme System

File: `theme.config.js` - Color Tokens

Defines all colors used in the app:

```
const themeColors = {
  primary: { light: '#1B5E20', dark: '#66BB6A' }, // Green (Mosher bl
  background: { light: '#ffffff', dark: '#0a0a0a' }, // Screen backgrou
  surface: { light: '#f8f8f8', dark: '#1a1a1a' }, // Card background
  foreground: { light: '#1a1a1a', dark: '#f5f5f5' }, // Text color
  muted: { light: '#616161', dark: '#9e9e9e' }, // Secondary text
  border: { light: '#d0d0d0', dark: '#2a2a2a' }, // Dividers
  success: { light: '#2E7D32', dark: '#81C784' }, // Success states
  warning: { light: '#E65100', dark: '#FFB74D' }, // Warnings
  error: { light: '#C62828', dark: '#EF5350' }, // Errors
};
```

Key Concept: Each color has `light` and `dark` variants. The theme system automatically switches between them based on device settings.

File: `lib/theme-provider.tsx` - Theme Context

Purpose: Manages light/dark mode globally

How It Works:

1. Detects System Theme

```
const systemScheme = useSystemColorScheme() ?? "light";
```

2. Applies Theme Across All Layers

```
applyScheme(scheme) {  
  nativewindColorScheme.set(scheme);           // NativeWind  
  Appearance.setColorScheme?.(scheme);         // React Native  
  document.documentElement.dataset.theme = scheme; // Web  
  // Set CSS variables for web  
  root.style.setProperty(`--color-${token}`, value);  
}
```

3. Exposes Theme Context

```
<ThemeContext.Provider value={{ colorScheme, setColorScheme }}>
```

File: `lib/_core/theme.ts` - Theme Builder

Purpose: Builds runtime theme palette

Exports:

- `Colors` - All color tokens organized by scheme
- `ThemeColorPalette` - TypeScript type for color object
- `ColorScheme` - Type for "light" | "dark"

File: `tailwind.config.js` - Tailwind Configuration

Purpose: Integrates Tailwind CSS with theme tokens

Key Features:

- Scans all `.tsx` files for Tailwind classes
- Converts theme tokens to Tailwind utilities
- Supports dark mode with CSS variables
- Enables NativeWind preset

Usage in Components:

```
// Light mode: bg-background = white
// Dark mode: bg-background = #0a0a0a
<View className="bg-background text-foreground p-4" />
```

File: `lib/utils.ts` - Utility Functions

`cn()` Function:

```
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

Purpose: Merge Tailwind classes safely without conflicts

Example:

```
cn(
  "p-4 rounded-lg",
  isActive && "bg-primary",
  disabled && "opacity-50"
)
// Result: "p-4 rounded-lg bg-primary opacity-50" (if both conditions true)
```

File: `components/screen-container.tsx` - SafeArea Wrapper

Purpose: Handles screen layout edge cases

What It Does:

- Extends background color behind status bar (notch)
- Keeps content within safe area (not under notch)
- Handles home indicator area (bottom of iPhone X+)
- Respects tab bar spacing

Usage:

```
<ScreenContainer className="p-4">  
  <Text>Screen content here</Text>  
</ScreenContainer>
```

Props:

- `className` - Tailwind classes for content
 - `edges` - Which edges to apply safe area to (default: top, left, right)
 - `containerClassName` - Classes for outer background container
-

Backend Architecture

File: `server/routers.ts` - API Routes

Purpose: Defines all tRPC API endpoints

Structure:

```

export const appRouter = router({
  health: publicProcedure.query(() => ({ status: "ok" })),

  employee: router({
    getProfile: protectedProcedure.query(...),
    createProfile: protectedProcedure.mutation(...),
    updateProfile: protectedProcedure.mutation(...),
  }),

  jobSites: router({
    list: protectedProcedure.query(...),
    create: protectedProcedure.mutation(...),
    update: protectedProcedure.mutation(...),
  }),

  timeLogs: router({
    getCurrentLog: protectedProcedure.query(...),
    clockIn: protectedProcedure.mutation(...),
    clockOut: protectedProcedure.mutation(...),
    list: protectedProcedure.query(...),
    getById: protectedProcedure.query(...),
    update: protectedProcedure.mutation(...),
    delete: protectedProcedure.mutation(...),
  }),
});

```

Key Concepts:

- **publicProcedure** - Accessible without authentication
- **protectedProcedure** - Requires authenticated user
- **query** - Read operation (GET)
- **mutation** - Write operation (POST/PUT/DELETE)

File: `server/_core/index.ts` - Server Startup

Purpose: Initializes Express server

What It Does:

1. Port Management

```
async function findAvailablePort(startPort = 3000)
// Tries ports 3000-3019 until one is available
```

2. CORS Configuration

```
// Allows requests from any origin with credentials
res.header("Access-Control-Allow-Origin", origin);
res.header("Access-Control-Allow-Credentials", "true");
```

3. Middleware Setup

```
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({ limit: "50mb" }));
```

4. Route Registration

```
registerStorageProxy(app);      // S3 file uploads
registerOAuthRoutes(app);      // Authentication
app.get("/api/health", ...);    // Health check
app.use("/api/trpc", ...);      // tRPC endpoints
```

File: `server/_core/oauth.ts` - Authentication Routes

Purpose: Handles OAuth login/logout

Routes:

- GET `/api/oauth/callback` - Web OAuth completion
- POST `/api/oauth/mobile` - Mobile token exchange
- POST `/api/auth/logout` - Clear session
- GET `/api/auth/me` - Get current user
- POST `/api/auth/session` - Token to cookie conversion

File: `server/db.ts` - Database Helpers

Purpose: Wraps Drizzle ORM queries

Key Functions:

- `getUserByOpenId()` - Fetch user by OAuth ID
- `upsertUser()` - Create or update user
- `getEmployeeByUserId()` - Fetch employee profile
- `createEmployee()` - Add employee
- `getActiveJobSites()` - List job sites
- `getCurrentTimeLog()` - Get active clock
- `clockIn()` - Create time log
- `clockOut()` - Complete time log

File: `drizzle/schema.ts` - Database Schema

Tables:

users

```
{
  id: int (auto-increment, primary key)
  openId: varchar (unique, from OAuth)
  name: text
  email: varchar
  loginMethod: varchar
  role: enum ('user', 'admin')
  createdAt: timestamp
  updatedAt: timestamp
  lastSignedIn: timestamp
}
```

employees

```
{
  id: int (auto-increment, primary key)
  userId: int (foreign key to users)
  employeeId: varchar (unique)
  name: varchar
  email: varchar
  phone: varchar
  role: enum ('employee', 'supervisor', 'admin')
  active: boolean
  createdAt: timestamp
  updatedAt: timestamp
}
```

jobSites

```
{
  id: int (auto-increment, primary key)
  name: varchar
  address: text
  status: enum ('active', 'inactive')
  createdAt: timestamp
  updatedAt: timestamp
}
```

timeLogs

```
{
  id: int (auto-increment, primary key)
  employeeId: int (foreign key)
  jobSiteId: int (foreign key)
  clockInTime: timestamp
  clockOutTime: timestamp (nullable)
  notes: text
  createdAt: timestamp
  updatedAt: timestamp
}
```

Configuration & Build

File: `app.config.ts` - Expo Configuration

Purpose: Configures the app for iOS and Android

Key Settings:

```
const env = {  
  appName: "Mosher Lawns Time Tracker",  
  appSlug: "mosher-lawns-time-tracker",  
  imageUrl: "", // S3 URL of app icon  
  scheme: schemeFromBundleId,  
  iosBundleId: bundleId,  
  androidPackage: bundleId,  
};
```

iOS Configuration:

- `bundleIdentifier` - Unique app ID (e.g., com.app.mosherlawnstimetracker)
- `supportsTablet` - Allow iPad support
- `infoPlist` - iOS-specific settings

Android Configuration:

- `adaptiveIcon` - Icon that adapts to device shape
- `package` - Android package name
- `permissions` - Required permissions (POST_NOTIFICATIONS)
- `intentFilters` - Deep linking configuration

Plugins:

- `expo-router` - File-based routing
 - `expo-audio` - Audio playback
 - `expo-video` - Video playback
 - `expo-splash-screen` - Splash screen
-

- `expo-build-properties` - Build settings

File: `package.json` - Dependencies

Scripts:

```
{
  "dev": "pnpm dev:server & pnpm dev:metro",
  "dev:server": "tsx watch server/_core/index.ts",
  "dev:metro": "expo start --web --port 8081",
  "build": "esbuild server/_core/index.ts --bundle --outdir=dist",
  "start": "node dist/index.js",
  "check": "tsc --noEmit",
  "test": "vitest run",
  "db:push": "drizzle-kit generate && drizzle-kit migrate"
}
```

Key Dependencies:

- `expo` - Development platform
- `react-native` - Mobile framework
- `react` - UI library
- `typescript` - Type checking
- `nativewind` - Tailwind CSS
- `@trpc/*` - Type-safe API
- `drizzle-orm` - Database ORM

Testing & Quality

File: `tests/auth.logout.test.ts` - Unit Tests

Purpose: Test authentication logout functionality

Framework: Vitest (similar to Jest)

Example Test:

```
test('logout clears session', async () => {  
  // Setup  
  // Execute  
  // Assert  
});
```

File: `eslint.config.js` - Code Linting

Purpose: Enforce code quality standards

Checks:

- TypeScript type correctness
- Unused variables
- Naming conventions
- Code style

Run: `pnpm lint`

File: `tailwind.config.js` - Tailwind Linting

Purpose: Ensure Tailwind classes are valid

Prevents:

- Invalid class names
 - Conflicting utilities
 - Unused styles
-

Data Flow Summary

Complete User Journey: Clocking In

1. **User** opens app
 - ↳ `app/_layout.tsx` runs
 - ↳ `initializeDefaultData()` seeds sample data
 - ↳ `ThemeProvider` wraps app
 - ↳ Stack navigation initialized
2. **User** navigates to **Clock In/Out** tab
 - ↳ `app/(tabs)/index.tsx` loads
 - ↳ `useEffect` fetches employees, jobs, active logs
 - ↳ Renders employee checkboxes **and** job dropdown
3. **User** selects Alice **and** Bob, selects "Site A"
 - ↳ `selectedEmployees = ["alice-id", "bob-id"]`
 - ↳ `selectedJobSite = "site-a-id"`
4. **User** taps "Clock In"
 - ↳ `handleClockIn()` **called**
 - ↳ **For each** employee:
 - ↳ `Storage.addTimeLog(employeeId, jobSiteId, Date.now())`
 - ↳ Creates `TimeLog` with `clockOutTime = null`
 - ↳ Saves to `AsyncStorage`
 - ↳ Refreshes **time** logs from storage
 - ↳ UI updates to **show** active clocks
5. **User** navigates to **Logs** tab
 - ↳ `app/(tabs)/logs.tsx` loads
 - ↳ Fetches **all time** logs
 - ↳ **Groups** logs by (`jobSiteId`, `clockInTime`, `clockOutTime`)
 - ↳ Renders grouped cards **with** employee badges
6. **User** taps "Clock Out" on the group
 - ↳ `handleClockOut()` **called**
 - ↳ **For each** log in group:
 - ↳ `Storage.clockOutTimeLog(logId)`
 - ↳ Sets `clockOutTime = Date.now()`
 - ↳ Saves to `AsyncStorage`
 - ↳ Refreshes UI

7. User taps "Export CSV"

- ↳ `handleExportLogs()` called
 - ↳ `Storage.exportLogsToCSV()`
 - ↳ Formats **all** logs **as** CSV string
 - ↳ Includes employee names, job sites, times, duration
 - ↳ Writes **to** device filesystem
 - ↳ Shows success message

Key Files Quick Reference

File	Purpose	Key Functions
<code>lib/local-storage.ts</code>	Data persistence	<code>addEmployee</code> , <code>getTimeLogs</code> , <code>exportLogsToCSV</code>
<code>app/_layout.tsx</code>	App initialization	Providers, splash screen, data seeding
<code>app/(tabs)/index.tsx</code>	Clock in/out	Multi-select, clock in/out logic
<code>app/(tabs)/logs.tsx</code>	View logs	Grouping, CSV export, edit/delete
<code>app/employees-management.tsx</code>	Manage employees	CRUD operations
<code>app/jobs-management.tsx</code>	Manage jobs	CRUD operations
<code>app/edit-log.tsx</code>	Manual entries	Multi-employee selection, date/time pickers
<code>lib/theme-provider.tsx</code>	Light/dark mode	Theme context, CSS variables
<code>theme.config.js</code>	Color tokens	Light/dark color definitions
<code>server/routers.ts</code>	API endpoints	tRPC routes (optional backend)
<code>drizzle/schema.ts</code>	Database schema	Table definitions (optional backend)

Summary

The **Mosher Lawns Time Tracker** is a well-structured React Native app with:

- **Clean Data Layer** - All logic centralized in `local-storage.ts`
- **Component-Based UI** - Reusable components with proper separation of concerns
- **Theme System** - Automatic light/dark mode support
- **Advanced Features** - Multi-employee selection, log grouping, CSV export
- **Optional Backend** - Full tRPC + Drizzle ORM setup for future scaling
- **Type Safety** - Full TypeScript throughout
- **Modern Stack** - Expo Router, NativeWind, React 19

The app prioritizes **simplicity and functionality** over complexity, making it easy to understand, maintain, and extend.